Standards de jetons:

l'evolution de web , s'explique simplemeent en observant ce que l'utilisateur peut faire a chaque etape:

web1: read-only

web2: Read-write: 2004 -aujourd'hui : le probleme de web2 : vos donnees appartiennent aux geant de la Tech(meta, Google), Si facebook decide de fermer votre compte, vous perdez tout.
+

pour gerez votre argent, ou vos achat sur internet vous devez passez par une banque.

web3 : Read-write-Own : remplacer **les intermediaire centraux par la Blockchain**.(une base de donnees decentrallisee partagee entre des milliers d'ordinateurs,) , ce que fait l'utilisateur ,il est veritablement proprietaire de ce qu;il possede sur internet,

Tableau comparatif pour bien comprendre la différence

Critère	**Web2 (Actuel)**
Où sont stockées les données ?	Sur les serveurs privés d'une entreprise (AWS, Google, Meta).
Comment s'identifie-t-on ?	Email + Mot de passe (ex: *Se connecter avec Google*).
Qui contrôle les jetons / l'argent ?	La banque ou la plateforme (ex: les crédits sur un jeu vidéo).
Règles du jeu	Décidées unilatéralement par l'entreprise.

dans le monde WEB2 : un solde est une entree dans une base de donnees privee geree par une unite centrale.

***token :** 1- token d'authentification (jetons de securite/API) : une chaine de caracteres securisee (une cle temporaire chiffree) generee par un serveur quand un utilisateur se

connecte.

2- token de paiment : Utilisé par les systèmes bancaires et les passerelles de paiement (Stripe, PayPal, Apple Pay). Lorsque vous enregistrez votre carte sur Amazon ou Apple Pay, l'entreprise ne conserve pas votre vrai numéro de carte sur ses serveurs (ce serait un risque de piratage énorme). Elle convertit ces données en un token bancaire. Si un pirate vole ce token Web2, il ne peut rien en faire en dehors du système qui l'a émis.

dans le monde WEB3 :

Une blockchain (ex: Ethereum, BNB Smart chain) est un registre public, décentralisé et immuable partagé par des milliers d'ordinateurs (noeuds).

Un token(jeton) : n'est pas un fichier physique ou une piece numerique telechargeable, C'est simplement une unite de compte geree par un programme informatique special appele un Smart Contract. (creer par un smart contract sur la blockchain)

Smart contrat : conserve une table (un tableau/dictionnaire) d'adresses et de soldes associes:

Adresse A --> 100 Tokens

Adresse B --> 50 Tokens

Coin vs Token :

***Coin (Cryptomonnaie native)** : L'unité monétaire de base de la blockchain (ex: BNB sur BNB Chain, ETH sur Ethereum). Servent à payer le Gas (les frais de transaction réseau).

token (jeton applicatif) : Un actif créé au-dessus d'une blockchain existante via un Smart Contract (ex: votre token 42Token). Pour transférer votre token, le réseau demande de payer des frais en coin native (BNB).

BlockChain :

NB:

-le reseau est constitue des ordinateurs participants.

-le blockChain est un registre partage et les regles qui organisent son evolution.

-Le smart Contrat est un programme execute sur cette infrastructure.

-Le site web est eventuellement l'interface qui permet a un humain de l'utiliser.

est une technologie qui permet d'enregistrer des informations dans un registre numerique entre plusieurs ordinateurs, de maniere a rendre ces informations difficiles a modifier retroactivement.

le mot signifie "chaine de blocs"

La différence essentielle avec une base de données classique est que la blockchain est généralement conçue pour fonctionner sans qu'une seule entreprise ou personne ait le contrôle absolu du registre.

Une blockchain est une base de données distribuée et sécurisée qui stocke l'historique de toutes les transactions échangées entre ses utilisateurs depuis sa création.

exemple:

tu veux envoyer 10 euro a un ami: une banque traditionnelle :

tu demande a ta banque de transferer de l'argent, la banque verifie ton compte, elle enregistre la transaction dans sa base de donnees , elle met a jour les soldes.

dans une blockChain comme Ethereum:

tu signe une transaction avec ta cle privee, tu transmets cette transaction au reseau , des participants du reseau verifie sa validite , **un mecanisme de consensus permet** d'integerer les transactions valides dans la blockChain , Les noeuds mettent a jour leur etat selon les regles du protocole.

NB :

La différence fondamentale est que le fonctionnement du système repose sur des règles cryptographiques et un protocole partagé, plutôt que sur une seule banque qui contrôle son registre.

La blockchain regroupe les transactions effectuées par les utilisateurs dans des conteneurs appelés Blocs. Chaque bloc contient trois éléments fondamentaux :

1-Les Données : La liste des transactions (ex: Alice envoie 2 BNB à Bob), les donnees technique,et une reference cryptographique au bloc precedent.

2-Son Hash : L'empreinte numérique unique du bloc (comme une empreinte digitale).

3-Le Hash du bloc précédent : Le lien mathématique qui relie ce bloc au bloc d'avant.

4-Les noeuds(nodes) : ce sont des ordinateurs qui participent au reseau. Il peuvent conserver les donnees, verifier les transactions et appliquer les regles de la blockchain.

5- Les cles cryptographiques : la cle privee sert notament a signer les transactions, la cle publc permet de verifier les signatures et contribue a l'identification cryptographique d'un compte.

6- Le consensus: C'est le mécanisme qui permet au réseau de s'accorder sur l'historique et l'état de la blockchain, selon les règles du protocole.

Si un pirate essaie de modifier une seule transaction dans le Bloc 1 :

*Le Hash du Bloc 1 change instantanément.

*Le Bloc 2 devient invalide car le "Hash précédent" qu'il contenait ne correspond plus.

*Tous les blocs suivants deviennent invalides.

*Les autres nœuds du réseau rejettent la modification du pirate.

NB

un exemple concret : creer ton propre token : (tokenizer project)

Tu veux que ton compte possède 1 000 FT42 et que tu puisses en envoyer 100 à Alice.

Voici ce qui se passe réellement:

1- tu ecris un smart contrat : tu programes en Slidity les regles du token: son nom ,ses soldes et ses transferts.

2- tu deploies le contrat sur une blockchain: tu envoie une transaction au reseau Ethereum , ou un reseau compatible.

3- le reseau enregistre le contrat: apres la validation du deploiement, le contrat possede une adresse et son etat est gere selon les regles du reseau

4- tu envoies 100 FT42 a alice : tu signes une transaction , le contrat verifie les conditions et , si le transfert est valide , met a jour les soldes.

Tu peux ensuite utiliser un portfeuille comme MetaMask pour afficher ton solde et evoyer des tokens.

---- Remix IDE , c'est l'outil que j'utilise pour travailler avec blockchain.

comparaison simplifiee 😀 :

Développement classique	**Développement blockchain**
VS Code	Remix IDE
Code Python	Code Solidity
Interpréteur ou environnement Python	EVM, qui exécute les smart contracts
Base de données	État conservé par la blockchain
Exécuter ton programme	Tester ou déployer ton smart contract

Token :

la blockchain est la technologie qui permet d'enregister et de verifier les operations, Le token est l'objet numerique que l'on cree ou que l'on echange grace a cette technologie.

blockChain : elle peut servir a gerer des billets , des poiints, des droits ou d'autres actif numeriques.

--> objectif : creer votre propre token et de le deployer sur une blockChain existante.

Étape 1 — Programmer le token

Tu écris un smart contract qui définit les règles de ton token : son nom, son symbole, sa quantité et les transferts autorisés.

Étape 2 — Créer les tokens

Par exemple, ton contrat crée 1 000 FT42 et les attribue à ton adresse. Ce sont des unités numériques gérées par le contrat, pas des pièces physiques.

Étape 3 — Déployer le contrat

Tu publies ton programme sur une blockchain compatible, par exemple un réseau de test Ethereum. Le contrat devient accessible sur ce réseau.

Étape 4 — Tester son fonctionnement

Tu vérifies que tu peux consulter ton solde, envoyer des tokens à une autre adresse et que les règles de ton contrat fonctionnent correctement.

NB :

un token ne represente pas forcement de l'argent reel , in faut distinguer trois choses.

A. Une cryptomonnaie : elle peut etre échangée contre des euros sur une plateforme qui propose cet échange. Son prix varie selon le marché.

B. Un token utilitaire

Exemple : 100 points numériques dans un jeu. Ils peuvent servir à acheter des objets dans le jeu sans pouvoir être échangés contre des euros.

C. Un token lié à une valeur monétaire

Exemple : un token conçu pour représenter un euro. Pour qu'il soit réellement échangeable contre un euro, il faut un mécanisme crédible de garantie et de remboursement. Le simple fait d'écrire « 1 token = 1 € » dans le code ne suffit pas.

Un token peut donc avoir une valeur d'usage, une valeur de marché ou une valeur liée à un actif. Ce sont trois notions différentes.

Le point essentiel : le token représente ici ton droit d'accès au concert, tandis que les 50 € constituent le paiement. Les deux peuvent être gérés par des systèmes différents.

Voici la distinction à retenir :

Élément	**Son rôle**
Blockchain	Le réseau et le registre partagé qui enregistrent les opé
Smart contract	Le programme qui définit les règles
Token FT42	L'actif numérique que ton contrat gère
Portefeuille	L'outil qui te permet d'utiliser ton adresse et de signer de
ETH	L'actif natif d'Ethereum, utilisable pour payer les frais et

Standards de jetons :

Les standards de jetons définissent la manière dont les jetons se comportent et interagissent à travers l'écosystème Ethereum

Imagine que tu veuilles créer ton token `GAME` et que tu veuilles que plusieurs personnes puissent consulter leurs soldes et échanger leurs tokens sans dépendre d'une seule base de données privée.

Tu peux utiliser une blockchain comme Ethereum.

Ethereum : Le réseau qui exécute les contrats et conserve l'état partagé

||

\ \ //

Smart contract : Le programme qui définit les règles de ton token

||

\ \ //

T**oken GAME** : Les soldes et les transferts sont gérés par le contrat

Un smart contract est un programme informatique qui s'exécute sur une blockchain compatible.

Tu peux le comparer à un distributeur automatique.

Tu lui donnes une instruction, il vérifie les règles, puis il exécute l'action autorisée.

Par exemple, ton contrat `GAME` peut définir ces règles :

1. Créer 1 000 tokens au départ.
2. Permettre au propriétaire d'envoyer des tokens à d'autres adresses.
3. Refuser un transfert si l'expéditeur ne possède pas suffisamment de tokens.
4. Permettre à une application de consulter un solde.

À quoi sert un standard de token comme ERC-20 ?

si chaque developer cree un token avec des fonctions differentes, un developer ecrit envoyer(), un autre ecrit transferer(). un autre utilise moveCoins().

Les applications auraient du mal a interagir avec tous ces tokens, c'est pourquoi Ethereum dispose de standards.

ERC-20 :

definit une interface commune pour les tokens interchangeable. elle comprend notamment les fonctions permattant de consulter les soldes, transferer les tokens et autoriser des depenses.

Creation de token :

tu ecris un smart contrat en Solidity:

<<Solidity : un langage de programation .>>

Tu écris un smart contract en Solidity :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract FT42 is ERC20 {
    constructor() ERC20("FT42", "FT42") {
        _mint(msg.sender, 1000 * 10 ** decimals());
    }
}
```

Voici ce que fait ce code :

Élément	Rôle
`FT42`	Le nom de ton contrat
`ERC20`	L'implémentation du standard utilisée
`"FT42"`	Le nom du token

`"FT42"`	Le symbole du token
`constructor()`	La fonction exécutée au déploiement
`_mint(...)`	Crée les tokens et les attribue à l'adresse qui déploie le contrat

Pour que votre token soit reconnu par des wallets (MetaMask, TrustWallet) et des explorateurs (BSCScan), votre Smart Contract doit suivre une convention de nommage standard : **BEP-20** (sur BNB Chain)ou **ERC-20** (sur Ethereum). Ces deux standards sont techniquement identiques.

1.ERC-20 : Les 6 Fonctions Obligatoires d'un ERC-20 / BEP-20

ERC-20 : definit des fonctions standard qui permettent aux applications compatibles d'utiliser les tokens de maniere uniforme.

on peut les diviser en quatre groupes:

- Consulter les informations du token.
- Consulter les soldes.
- Transférer des tokens.
- Autoriser un autre compte ou contrat à dépenser des tokens.

4.1. Les fonctions d'information

`name()`

Retourne le nom du token.

```
name(); // "FT42"
```

`symbol()`

Retourne son symbole.

```
symbol(); // "FT42"
```

`decimals()`

Retourne le nombre de décimales d'affichage. Dans l'implémentation OpenZeppelin habituelle, il vaut 18 par défaut.

```
decimals(); // 18
```

`totalSupply()`

Retourne la quantité totale de tokens en circulation selon le contrat.

```
totalSupply();
```

Dans notre exemple initial, cela correspond à 1 000 FT42 affichés, soit 1000×10^{18} unités minimales.

4.2. : consulter un solde : `balanceOf(address)`

C'est la fonction qui répond à la question : « Combien de tokens cette adresse possède-t-elle ? »

```
balanceOf(alice);
```

Imaginons ces soldes :

Adresse	Solde
Toi	1 000 FT42
Alice	0 FT42
Bob	0 FT42

Si tu appelles `balanceOf` avec ton adresse, le contrat retourne le montant associé à ton adresse.

Cette fonction est une lecture : elle ne modifie pas les soldes.

4.3. `transfer(to, amount)` : envoyer des tokens

C'est la fonction qui permet au propriétaire de transférer ses propres tokens.

```
transfer(alice, 100 * 10 ** 18);
```

Elle signifie : « Envoie 100 FT42 à Alice », dans notre exemple à 18 décimales.

Si tu possèdes 1 000 FT42, après le transfert :

- ton solde devient 900 FT42 ;
- celui d'Alice devient 100 FT42 ;
- le total reste égal à 1 000 FT42.

Le contrat vérifie notamment que tu possèdes suffisamment de tokens.

Sur une blockchain réelle, l'appel modifiant les soldes doit être envoyé comme une transaction et nécessite normalement des frais de gas.

--> security : n importe quelle persoonne peux acceder a voir le contrat - > mint create token dois etre securise par ownerOnly .

----------nb ----------

---

1. Le premier `_mint` (dans le `constructor`) : La création initiale

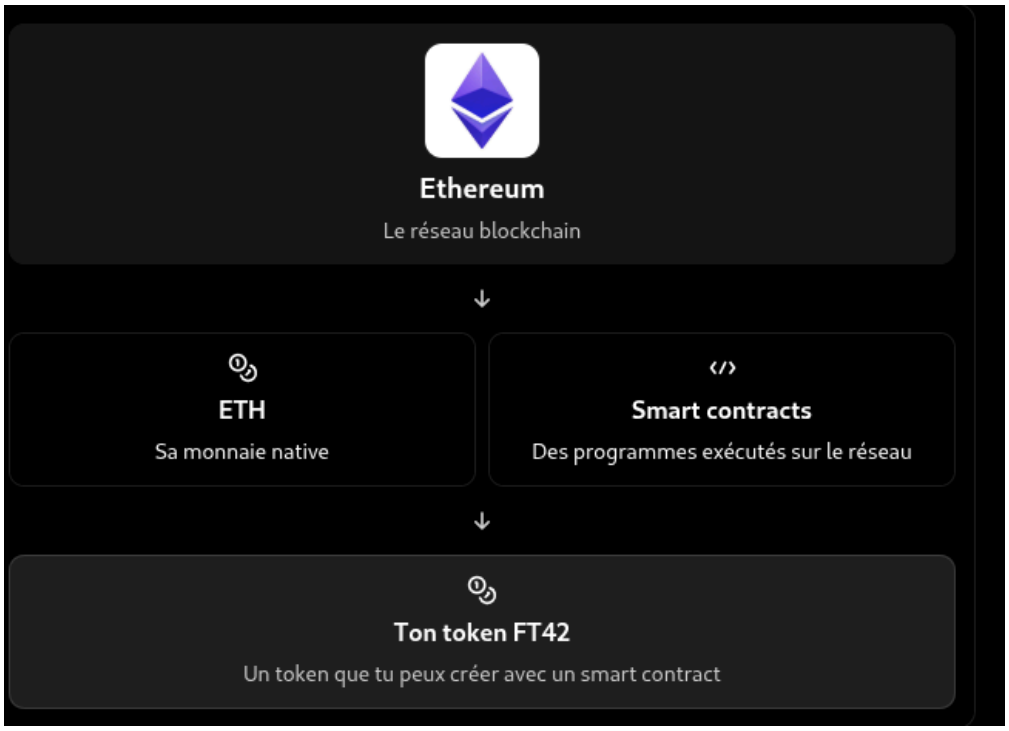
- **Quand ?** Il s'exécute **une seule et unique fois**, au moment exact où vous déployez le contrat sur la blockchain.
- **À quoi il sert ?** Il permet de générer le stock de départ de vos tokens (le *supply* initial) et de les déposer directement dans votre portefeuille (`msg.sender`). C'est grâce à ça que vous possédez vos premiers tokens dès que le contrat est validé.

2. Le deuxième `_mint` (dans la fonction `mint` en dehors) : La création future (optionnelle)

- **Quand ?** Il peut s'exécuter **plus tard**, à n'importe quel moment après le déploiement, *uniquement* si vous (le propriétaire) décidez d'appuyer sur le bouton `mint` pour fabriquer de nouveaux tokens.
- **À quoi il sert ?** Il donne la flexibilité d'augmenter la quantité de tokens en circulation dans le futur (par exemple si votre projet grandit et que vous avez besoin de distribuer ou vendre de nouveaux jetons).

Ethereum 😀:

est une blockchain qui permet non seulment d'envoyer de l'argent numerique, mais aussi d'executer des programmes appeles smart contrats.



Terme	Signification
Ethereum	Le réseau blockchain et la plateforme
ETH	La monnaie native utilisée sur ce réseau

FT42	Ton propre token, géré par ton smart contract
Remix IDE	L'outil dans lequel tu écris et déploies ton contrat

- thereum est comme le pays et son infrastructure.
- ETH est comme la monnaie utilisée dans ce pays.
- FT42 est comme une unité numérique que tu crées dans ce système, avec ses propres règles.

Voici ce qui se passe :

1. Tu écris ce programme dans Remix.
2. Tu le compiles pour vérifier qu'il est valide.
3. Tu le déploies sur Ethereum, ou d'abord sur un environnement de test.
4. Le contrat gère les 1 000 FT42 et les transferts selon ses règles.

Tu n'as donc pas besoin de créer ta propre blockchain : tu utilises Ethereum comme infrastructure pour faire fonctionner ton token.

Solidity : OOP , high level , used to write smart contracts.

inspired by Javascript, C++, pyhton.

est un langage de programmation, il est notamment conçu pour écrire des smart contracts qui s'exécutent sur des blockchains compatibles avec la machine virtuelle Ethereum (EVM).

code Solidity (ybouzafo42.sol) -> compilateur Solidity (transforme le code en bytecode compatible EVM) -> Blockchain compatible EVM (le contrat est deployer et ses fonctions peuvent s'executees).

pourquoi le compiler parce que le reseau ne prend pas directement ton fichier source solidity comme programme a executer. Le compilateur produit notamment du bytecode que l'EVM peux executer.

Solidity is statically typed , supports inheritance, librairie,and complex user-defined types, among other feautures.

-> la particularite de Solidity est d'etre concu specifiquement pour etre execure sur l'EVM(Ethereum Virtual Machine) .

Structure minimale d'un fichier :

tous le fichier Solidity commence par deux lignes obligatoires, suivies de la declaration du contrat(contrat) 😀

Solidity

```
// 1. Déclaration de la licence Open Source (obligatoire pour le compilateur)
// SPDX-License-Identifier: MIT

// 2. Version du compilateur à utiliser (Pragma Directive)
pragma solidity ^0.8.20;

// 3. Le bloc du contrat (équivalent à 'class' en Java ou C++)
contract MonPremierContrat {
    // Le code du contrat vient ici !
}
```

2. Les Variables & Les Types de Données

Contrairement a JavaScript ou Python ou les types sont dynamiques, en Solidity. il faut declarer le type de chaque variable.

```
contract TypesDemo {
    // 1. Entier non-signé (positif uniquement) : uint256 ou uint (0 à 2^256 - 1)
    uint256 public nombrePositif = 42;

    // 2. Entier signé (positif ou négatif) : int256 ou int
    int256 public nombreSigne = -10;
```

```
        // 3. Booléen : true ou false
        bool public estActif = true;

        // 4. Chaîne de caractères
        string public message = "Hello 42";

        // 5. Adresse blockchain (format 0x...)
        address public monAdresse = 0x1234567890123456789012345678901234567890;
    }
```

B. Le type spécial : `mapping` (Dictionnaire / Table de hachage)

C'est le composant le plus important pour gerer des soldes dans un token. Un **mapping** associe une cle a une valeur.

```
    // Assigne à chaque adresse un solde numérique (uint256)
    mapping(address => uint256) public balances;

    // Exemple de contenu sous-jacent :
    // 0xAAA... -> 50
    // 0xBBB... -> 100
```

3. Les Fonctions (Syntaxe & Visibilité)

En Solidity la definition d'une fonction requiert de preciser sa vibilite(qui peut l'appeler) et son comportement par rapport aux donnees.

La syntaxe globale :

Plaintext

```
    function nomDeLaFonction(type param1) [visibilité] [comportement] returns
    (typeRetour) {
        // corps de la fonction
    }
```

A .Les visibilites (public, private , internal, external) 😀 :

- **`public`** : Appelable depuis l'intérieur ET l'extérieur du contrat.
- **`private`** : Appellable **uniquement** à l'intérieur de ce contrat spécifique.
- **`internal`** : Accessible par ce contrat et ceux qui en héritent.
- **`external`** : Appellable **uniquement** depuis l'extérieur du contrat.

B. Le Comportement (`view` , `pure` , ou rien)

- **`view`** : La fonction **lit** des variables sur la blockchain sans les modifier (appel gratuit en gas).
- **`pure`** : La fonction **ne lit NI ne modifie** l'état de la blockchain (ex: faire un calcul pur comme $2 + 2$).
- *(Rien)* : La fonction **modifie l'état** de la blockchain (coûte du gas).

- *(Rien)* : La fonction **modifie l'état** de la blockchain (coûte du gas).


```
contract FonctionsDemo {
    uint256 public x = 10;

    // Modification d'état (payant en gas)
    function modifierX(uint256 nouveauX) public {
        x = nouveauX;
    }

    // Lecture d'état (gratuit)
    function lireX() public view returns (uint256) {
        return x;
    }

    // Calcul pur sans accès à la mémoire de la blockchain (gratuit)
    function additionner(uint256 a, uint256 b) public pure returns (uint256) {
        return a + b;
    }
}
```

4.Les Modificateurs de Securite (require & modifier)

A. La vérification de condition : `require`

`require(condition, "Message d'erreur")` sert à valider une instruction. Si la condition est fausse, le contrat annule immédiatement la transaction.

```
function transferer(address destinataire, uint256 montant) public {
    // Annule la transaction si le solde est inférieur au montant demandé
    require(balances[msg.sender] >= montant, "Solde insuffisant !");

    balances[msg.sender] -= montant;
```

```
        balances[destinataire] += montant;
    }
```

B. Le `modifier` (Réutilisation de règles de sécurité)

Un `modifier` est un bloc de code réutilisable attaché à une fonction pour restreindre son accès. Le symbole `_;` indique l'endroit où le code de la fonction s'exécutera si la condition passe.

Solidity

```
address public owner;

constructor() {
    owner = msg.sender; // msg.sender est celui qui déploie le contrat
}

// Définition du modificateur
modifier onlyOwner() {
    require(msg.sender == owner, "Accès refusé : vous n'êtes pas le propriétaire !");
    _; // Poursuit l'exécution de la fonction
}

// Utilisation du modificateur
function fonctionSecrete() public onlyOwner {
    // Seul 'owner' peut exécuter ce code
}
```

Synthèse : Un exemple complet commenté

Voici un contrat récapitulatif qui rassemble tous ces concepts :

Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MiniBank {
    // 1. State Variables (Variables d'état)
    address public owner;
    mapping(address => uint256) public balances;

    // 2. Modifier
    modifier onlyOwner() {
```

```
            require(msg.sender == owner, "Seul le propriétaire peut exécuter
    cela");
            _;
        }

        // 3. Constructor
        constructor() {
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MiniBank {
    // 1. State Variables (Variables d'état)
    address public owner;
    mapping(address => uint256) public balances;

    // 2. Modifier
    modifier onlyOwner() {
        require(msg.sender == owner, "Seul le propriétaire peut exécuter cela");
        _;
    }

    // 3. Constructor
    constructor() {
        owner = msg.sender;
    }

    // 4. Fonctions
    function deposer() public payable {
        // msg.value contient la quantité de BNB/ETH envoyée avec la transaction
        require(msg.value > 0, "Montant invalide");
        balances[msg.sender] += msg.value;
    }

    function consulterSolde(address compte) public view returns (uint256) {
        return balances[compte];
```

```
    }
}
```

Questions:

1.pragmas : **pragma** vient du grec *pragma* (action, affaire). En programmation (et particulièrement en Solidity), c'est une **directive destinée au compilateur**.

- Elle lui indique **comment** il doit traiter le code source.
- *Exemple :* `pragma solidity ^0.8.20;` dit au compilateur : *"Utilise la version 0.8.20 de Solidity (ou une version compatible légèrement supérieure) pour traduire ce code en bytecode compréhensible par la blockchain"*. Si vous essayez de compiler avec une version trop ancienne, le compilateur refusera.

2. **visibilités:** **i**l faut separer le monde "interne" (le code a l'interieur du Smart contract) et le monde "externe" (les utilisateurs, les sites web , les wallets comme MetaMask , ou d'autres Smart Contracts).

   *__L'intérieur (Interne) :__ C'est l'exécutable du contrat lui-même. Les autres fonctions écrites à l'intérieur du même contrat peuvent appeler ces fonctions ou lire ces variables.

   *__L'extérieur (Externe) :__ C'est tout ce qui se passe en dehors de la blockchain ou via des transactions de l'extérieur. Par exemple, lorsque vous ouvrez une application web (Frontend) ou l'interface de Remix ou de BSCScan pour cliquer sur un bouton ou appeler une fonction, vous agissez de l'extérieur

3.les gas :

Le **Gas** (carburant en anglais) représente les **frais de calcul** payés sur la blockchain.

- Contrairement à un serveur Web2 (où l'hébergeur paie l'électricité et les serveurs), sur la blockchain, chaque ligne de code exécutée (additionner deux nombres, modifier une variable, stocker des données) demande de l'énergie à des milliers d'ordinateurs (les validateurs/nœuds).
- Pour éviter que quelqu'un écrive une boucle infinie qui ferait planter tout le réseau, **chaque opération coûte une quantité précise de Gas**.
- Vous payez ces frais en monnaie native (par exemple en `tBNB` sur le testnet de la BNB Chain) à chaque fois que vous envoyez une transaction qui modifie l'état de la blockchain.

4.balances:

Dans un token, balances est un dictionnaire(un mapping) qui associe une adresse de portefeuille a un solde de jetons.

balances = {

0x111...AAA (votre wallet) : 500 tokens,

0x222...BBB (un ami) : 100 tokens

}

5.Ownable(msg.sender) vs owner :

- Vous n'avez pas besoin de reecrire manuellement modifier onlyOwner, ou la variable owner, car **OpenZeppelin le fait déjà pour vous** en arrière-plan lorsque vous écrivez `is Ownable`.
- `Ownable(msg.sender)` est simplement l'appel au **constructeur du contrat** **`Ownable`** fourni par OpenZeppelin. Il lui transmet l'adresse `msg.sender` (celui qui déploie le contrat) pour qu'il devienne automatiquement le propriétaire officiel du contrat. C'est pour cela que le modificateur `onlyOwner` devient immédiatement disponible et fonctionnel sans que vous ayez à le coder vous-même.

6. Dans `function deposer() public payable { ... }`, c'est quoi `payable` ?

Par défaut en Solidity, lorsqu'une fonction est appelée, elle ne peut **pas** recevoir d'argent natif (comme des BNB ou des ETH).

Le mot-clé **`payable`** est une autorisation explicite :

- Il indique que cette fonction **a le droit d'accepter des fonds** envoyés en même temps que la transaction.
- Si un utilisateur appelle la fonction `deposer()` en y attachant 0.1 BNB, le mot-clé `payable` permet au contrat d'encaisser cet argent et de l'enregistrer (via par exemple `msg.value`, qui représente la quantité de BNB envoyée).

7.bytecode :

le blockchain n'est pas un serveur web traditionnel: c'est un reseau geant compose de milliers d'ordinateurs independants(les noeuds) qui doivent executer le meme code en

meme temps pour se mettre d'accord, le bytecode y joue un role vital pour plusieurs raisons:

- **L'EVM (Ethereum Virtual Machine) ne comprend que ça :** Les nœuds de la blockchain exécutent les contrats intelligents via une machine virtuelle (l'EVM). Cette machine ne sait pas lire du texte en Solidity. Elle a besoin d'un format universel, compact et standardisé : le bytecode.
- **La Sécurité et le Déterminisme :** Peu importe que le code source original ait été écrit en Solidity, Vyper ou un autre langage. Une fois compilé en bytecode, il s'exécute de manière **strictement identique** sur l'ordinateur d'un validateur au Japon, d'un autre au Brésil ou d'un autre en France. Cela garantit qu'il n'y a pas d'ambiguïté dans le calcul.
- **Le stockage sur le registre (La Blockchain) :** Lors de la phase de déploiement (quand vous cliquez sur "Deploy" dans Remix), vous envoyez ce bytecode sur la blockchain. Il est stocké de façon permanente et immuable dans une transaction. C'est ce bytecode qui définit votre Smart Contract de manière définitive.

Les noeuds de blockchain executent les contrats intelligents via une machine virtuelle (l'EVM)

La Machine Virtuelle (EVM)

est le "moteur de calcul" ou le " cerveau" décentralisé qui fait tourner tous les Smart Contracts sur les blockchains compatible

Qu'est ce que l'EVM concretement ? : imaginons un immense ordinateur virtuel mondial. Cet ordinateur n'appartient a personne en particulier, mais il est execute simultanement par tous les noeuds (les ordinateurs du reseau) de blockchain a travers le monde.

- L'EVM est le logiciel installé sur chaque nœud qui permet d'exécuter le **bytecode** de vos Smart Contracts.
- Peu importe que votre nœud tourne sur un ordinateur sous Linux en Europe, un Mac en Asie ou un PC en Afrique : l'EVM garantit que le code s'exécutera de la **même manière exacte**, partout, sans aucune déviation

2. Comment fonctionne l'EVM ? (Le cycle d'exécution)

Le parcours d'un Smart Contract à travers l'EVM se fait en quatre grandes étapes :

1. **L'écriture (Solidity) :** Vous écrivez votre code de token en langage humain Solidity (ex: `Token42.sol`).

2. **La Compilation :** Le compilateur (via Remix ou Hardhat) traduit votre code Solidity en **bytecode** (du code hexadécimal de bas niveau) et génère l'**ABI** (l'interface de communication).
3. **Le Déploiement :** Vous envoyez ce bytecode sur la blockchain via une transaction de déploiement. L'EVM enregistre ce bytecode à une **adresse de contrat unique** (ex: `0x1234...ABCD`) de manière permanente et immuable.
4. **L'Exécution (Appel de fonction) :** Lorsqu'un utilisateur clique sur "Transfer" ou "Mint", il envoie une transaction contenant des données à cette adresse. L'EVM de chaque nœud du réseau charge le bytecode de ce contrat, l'exécute localement, met à jour l'état de la blockchain, et valide le résultat par consensus.

Pourquoi l'EVM est-elle révolutionnaire ?

1. **Le Déterminisme absolu :** Dans la programmation classique (Web2), si vous exécutez un script sur un serveur, il peut dépendre de l'heure du système, d'une base de données externe ou d'une API qui peut changer. L'EVM est **totalement déterministe** : pour un même état de la blockchain et des données d'entrée identiques, l'EVM produira *toujours* exactement le même résultat, peu importe l'ordinateur qui l'exécute.
2. **Le Sandbox (Environnement isolé) :** Les Smart Contracts s'exécutent dans une "bulle" sécurisée. Un contrat malveillant ne peut pas accéder aux fichiers de votre ordinateur personnel ni pirater d'autres applications, car l'EVM isole complètement l'exécution du code.
3. **La gestion du Gas (Le carburant anti-plantage) :** Puisque l'EVM n'a pas de gestionnaire central, comment évite-t-on qu'un développeur écrive une boucle infinie (`while(true)`) qui bloquerait tous les ordinateurs de la planète ? C'est là que le **Gas** intervient : chaque instruction exécutée par l'EVM consomme une quantité précise de carburant (de Gas). Si le quota de gas payé par l'utilisateur est épuisé en plein milieu d'une boucle infinie, l'EVM arrête brutalement l'exécution, annule la transaction (*revert*), et libère le réseau.

LE MÉCANISME DE CONSENSUS:

Puisqu'il n'y a pas de patron central pour valider les blocs, comment le réseau se met-il d'accord sur la version exacte des données ? C'est le rôle du mécanisme de consensus.

1. Proof of Work (PoW) — *Preuve de Travail* (ex: Bitcoin)

Les ordinateurs (les "mineurs") s'affrontent pour résoudre un problème mathématique complexe et très gourmand en énergie. Le premier qui trouve la solution gagne le droit d'ajouter le bloc et reçoit une récompense.

2. Proof of Stake (PoS) / Proof of Staked Authority (PoSA) — (ex: BNB Chain)

Au lieu de consommer de l'électricité, les validateurs bloquent (mettent en "stake") une certaine quantité de tokens pour prouver leur bonne foi. S'ils essaient de tricher, ils perdent leurs tokens bloqués. C'est ce système ultra-rapide et économique qui est utilisé sur la **BNB Smart Chain**.

??? ecors ps compris

LES SMART CONTRACTS 😀***:***

Un Smart Contract est un programme informatique autonome stocké directement sur la blockchain, qui s'exécute automatiquement lorsque des conditions prédéfinies sont remplies, sans aucun intermédiaire humain.

Introduction to Smart Contracts:

commencons par un exemple simple qui definit la valeur d'une varible et la rend accessible a d'autres contrats.

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.4.16 <0.9.0;
```

```
contract SimpleStorage {
        uint storedData;

        function set(uint x) public {
        storedData = x;
        }

        function get() public view returns (uint) {
        return storedData;
}
}
```

1* La première ligne indique que le code source est soumis à la licence GPL version 3.0. Les spécificateurs de licence lisibles par machine sont importants dans un contexte où la publication du code source est la règle par défaut.
2*La ligne suivante précise que le code source est écrit pour la version 0.4.16 de Solidity, ou pour une version plus récente du langage allant jusqu'à la version 0.9.0, non comprise. Cela permet de s'assurer que le contrat ne sera pas compilable avec une nouvelle version (incompatible) du compilateur,
3* Les **Pragmas** sont des instructions courantes destinées aux compilateurs, qui leur indiquent comment traiter le code source (par exemple, « pragma once »).

Un contrat au sens de solidity est un ensemble de code (ses fonctions) et de donnees (son etat) qui reside a une adresse specifique sur blockchain Ethereum.

*La ligne `uint storedData;` déclare une variable d'état nommée `storedData` de type `uint` (entier non signé de 256 bits). On peut la considérer comme un emplacement unique dans une base de données que l'on peut interroger et modifier en appelant les fonctions du code qui gère

cette base de données. Dans cet exemple, le contrat définit les fonctions set et

resume :

-> deploy contrat : envoyer definitivement le code(le bytecode) sur le blockchain : avant deployement le code existe sauf sur votre ordinateur ou dans remix. il ne fait rien de reel.

Pendant deployement : Vous envoyez une transaction spéciale sur le réseau. La blockchain valide cette transaction et enregistre votre code de manière permanente à une adresse unique (ex: 0x123...).

apres deployement : Le contrat devient un programme autonome qui vit sa vie sur la blockchain. Personne ne peut plus le modifier, et tout le monde peut interagir avec lui à distance.

⚙️ *Le Compilateur (Le Traducteur)*

Son rôle : Solidity (votre code lisible par un humain) ne peut pas être lu directement par la machine virtuelle de la blockchain (l'EVM). Le compilateur prend votre code Solidity et le traduit en bytecode (du code binaire/hexadécimal pur).

Sa relation : Il est intégré directement dans Remix IDE. Quand vous cliquez sur "Compile", il transforme votre code source pour qu'il soit prêt à être envoyé.

🦊 *MetaMask (Le Portefeuille & le Garde du corps):*

Son rôle : C'est votre application/extension de portefeuille. Il stocke vos clés privées (vos mots de passe secrets) et signe numériquement vos actions.

Sa relation : Lorsque vous voulez déployer votre contrat depuis Remix, Remix demande à MetaMask : "Est-ce que l'utilisateur est d'accord pour payer et envoyer ce contrat ?". MetaMask s'ouvre, vous demande votre accord, et signe la transaction.

🚰 Le Faucet (Le Distributeur d'argent gratuit)

Son rôle : Sur un réseau de test (comme Sepolia), les transactions coûtent du Gas (des frais). Le Faucet est un site web gratuit qui vous donne de la "fausse" monnaie (des ETH de test) pour pouvoir payer ces frais sans utiliser votre vrai argent.

Sa relation : Vous l'utilisez une seule fois avant de commencer pour alimenter votre portefeuille MetaMask en monnaie de test.

🌐 Sepolia / Testnet (Le Terrain d'entraînement)

Son rôle : C'est un clone de la vraie blockchain Ethereum, mais qui sert uniquement aux tests. L'argent y est virtuel et sans valeur.

Sa relation : C'est la destination finale. C'est sur Sepolia que MetaMask envoie votre bytecode compilé par Remix. Une fois déployé sur Sepolia, votre contrat y vit pour de vrai.

en Gros 😀 ========

Le but fondamental du projet **Tokenizer** est de :

1. **Créer** vos propres tokens numériques via un Smart Contract Solidity (en respectant la règle d'avoir le chiffre `42` dans le nom).
2. **Déployer** ce contrat sur une blockchain de test (comme Sepolia ou BSC Testnet).
3. **Les manipuler** (notamment les transférer d'un portefeuille à un autre, vérifier les soldes, et tester les fonctions de sécurité comme le `mint` ou le `onlyOwner`).

En gros, vous devenez le créateur de votre propre monnaie numérique décentralisée, et vous prouvez son bon fonctionnement lors de la soutenance en effectuant des manipulations en direct !

me:

le contrat ,sert a creer des tokens , mais pour deployer ce contrat on a besoin de frais, donc on prend de l'argent fausse de faucet , et on les mets dans mon Wallet metamask , et apres , quaand je veux compiler le contrat dans remixIDE , apres deploy , en choisissant metamask qui pais pour valider la transaction, , ensuite cette contrat est partie a spoila qui est comme une blockchain, et enregister ,

==========NB==========

notions critique de decimale:

en EVM : les nombres decimaux n'existent pas pour eviter les erreurs d'arrondi sur les ordinateurs du reseau.

- EVM fait une division entiere par un arrondi classic : supprime la partie decimale.

Pourquoi ? Pour que les calculs suivent des règles précises et déterministes : tous les nœuds qui exécutent le contrat doivent obtenir exactement le même résultat. Les nombres à virgule flottante peuvent introduire des différences d'arrondi selon les calculs et les représentations numériques.

2. Alors, comment représenter `1,5` si les décimales ne sont pas disponibles ?

La solution est simple : on stocke un nombre entier plus grand, puis on interprète ce nombre avec une échelle.

Imagine que tu veuilles représenter des euros en centimes.

Exemple : l'euro et le centime

Montant affiché

1,50 €

Montant stocké

150 centimes

On ne stocke pas directement `1,50`. On stocke l'entier `150`, puis on sait qu'il faut diviser par `100` pour retrouver les euros.

\[150 \div 100 = 1,50 \]

C'est le même principe qui est utilisé pour les tokens ERC-20 !

. Comprendre `decimals` avec un token ERC-20

Dans un token ERC-20, la fonction `decimals()` indique combien de chiffres après la virgule on doit afficher.

La valeur par défaut utilisée par OpenZeppelin est généralement :

```
function decimals() public view virtual returns (uint8) {
    return 18;
}
```

Cela signifie que le token utilise une échelle de 10^18.

1 token = 10^{18} unitee internes

Imaginons que tu crées ton token `FT42` avec 18 décimales.

Comment les quantités sont représentées

Quantité affichée	Valeur entière stockée

1 FT42	`1000000000000000000`
2 FT42	`2000000000000000000`
0,5 FT42	`500000000000000000`
0,01 FT42	`10000000000000000`

Attention : la blockchain stocke et manipule les unités entières. L'interface du portefeuille convertit ces unités en quantités lisibles pour toi.

4. Pourquoi utilise-t-on 18 décimales ?

Ce choix permet de représenter des fractions très petites d'un token.

Par exemple, si ton token a 18 décimales, tu peux représenter une quantité aussi petite que :

0,000000000000000001 FT42

Cela correspond à une seule unité interne.

5. Le point le plus important pour ton projet Tokenizer

Regarde ce code Solidity :

```
contract FT42 is ERC20 {
    constructor() ERC20("FT42", "FT42") {
        _mint(msg.sender, 1000 * 10 ** decimals());
    }
}
```

Décomposons cette ligne :

```
_mint(msg.sender, 1000 * 10 ** decimals());
```

1. `1000` représente le nombre de tokens que tu veux créer.
2. `decimals()` renvoie `18` par défaut.
3. `10 ** decimals()` signifie 10^18.
4. On multiplie `1000` par 10^18 pour convertir les tokens en unités internes.
5. `_mint` attribue ces unités au portefeuille qui déploie le contrat, c'est-à-dire `msg.sender` dans ce constructeur.

Le résultat interne est :

```
1000000000000000000000
```

Mais ton portefeuille affichera normalement :

1 000 FT42, et non un nombre avec 21 chiffres !

Sans la multiplication par `10 ** decimals()`, tu ne créerais que 1 000 unités internes, soit seulement `0,000000000000001 FT42` avec 18 décimales

TODO:

1-les fonction obligatoire d'un ERC-20 --> .....

2- les events obligatoires -> .....

3- notion critique de decimale (nb) -> done hamdolah

4- secutity -> .....

5- mecanisme de consensus ->.....